```
import React, {
  forwardRef,
  useState,
  useEffect,
  useRef,
  useCallback,
  useMemo,
} from "react";
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";
import { ArrowUp, ArrowDown, ArrowUpDown } from "lucide-react";

// ◍ THE SACRED OFFICIAL TANSTACK CORE & VIRTUAL IMPORTERS SECURED!
import {
  createTable,
  type ColumnDef,
  type RowSelectionState,
  type SortingState,
  type ColumnSizingState,
  type Row,
  type Column,
  getCoreRowModel,
  getSortedRowModel,
  getFilteredRowModel,
} from "@tanstack/table-core";
import { useVirtualizer } from "@tanstack/react-virtual";

import { TextBox } from "../data-entry/text-box";
import { NumericEdit } from "../data-entry/numeric-edit";
import { DateEdit } from "../data-entry/date-edit";
import { PremiumCheckbox } from "../selector/checkbox";
import { ComboBox } from "../selector/combo-box";
import { HintBox } from "../feedback/hint-box";
import { StandalonePagination } from "../feedback/standalone-pagination";

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

export interface PopupMenuConfig {
  trigger: (e: React.MouseEvent, recordRow?: Record<string, unknown>) => void;
  [key: string]: unknown;
}

export interface GridColumnLayout {
  key: string;
  headerLabel: string;
  widthPercent: number;
  align?: "left" | "center" | "right";
  editType?: "text" | "numeric" | "date" | "checkbox" | "combo";
```

```
  editOptions?: Record<string, unknown>[];
  enableSorting?: boolean;
  enableFiltering?: boolean;
}

export interface GridDataChangePayload {
  rowIndex: number;
  columnKey: string;
  oldValue: unknown;
  newValue: unknown;
}

export interface GridProps extends Omit<
  React.HTMLAttributes<HTMLDivElement>,
  "onChange"
> {
  label?: string;
  error?: string;
  hint?: string;
  popupMenu?: PopupMenuConfig;
  columns: GridColumnLayout[];
  data: Record<string, unknown>[];
  showSelectionCheckbox?: boolean;
  isEditable?: boolean;
  isRowMovable?: boolean;
  isColumnMovable?: boolean;
  rowIdentifierKey?: string;
  rowHeightPx?: number;
  activeRowKey?: string;
  disabled?: boolean;
  onRowClick?: (row: Record<string, unknown>, index: number) => void;
  onDataChange?: (
    updatedData: Record<string, unknown>[],
    changedInfo: GridDataChangePayload,
  ) => void;
  onRowMove?: (fromIndex: number, toIndex: number) => void;
  onColumnMove?: (fromIndex: number, toIndex: number) => void;
}

export const Grid = forwardRef<HTMLDivElement, GridProps>(
  (
    {
      label,
      error,
      hint,
      popupMenu,
      columns = [],
      data = [],
      showSelectionCheckbox = false,
      isEditable = false,
```

```
    isRowMovable = false,
    isColumnMovable = false,
    rowIdentifierKey = "id",
    rowHeightPx = 42,
    activeRowKey,
    onRowClick,
    onDataChange,
    onRowMove,
    onColumnMove,
    className,
    ...props
  },
  ref,
) => {
  const parentRef = useRef<HTMLDivElement>(null);
  const tableRef = useRef<HTMLTableElement>(null);
  const [sorting, setSorting] = useState<SortingState>([]);
  const [columnFilters, setColumnFilters] = useState<
    import("@tanstack/table-core").ColumnFiltersState
  >([]);
  const [globalFilter, setGlobalFilter] = useState<string>("");
  const [columnOrder, setColumnOrder] = useState<string[]>([]);
  const [columnVisibility, setColumnVisibility] = useState<
    Record<string, boolean>
  >({});
  const [rowSelection, setRowSelection] = useState<RowSelectionState>({});
  const [editingCell, setEditingCell] = useState<{
    rowIndex: number;
    columnKey: string;
  } | null>(null);
  const [editValue, setEditValue] = useState<unknown>("");
  const [columnSizing, setColumnSizing] = useState<ColumnSizingState>({});
  const [isResizing, setIsResizing] = useState<{
    columnKey: string;
    startX: number;
    startWidth: number;
  } | null>(null);
  const [draggedColumn, setDraggedColumn] = useState<string | null>(null);
  const [draggedRow, setDraggedRow] = useState<number | null>(null);
  const [filterInputs, setFilterInputs] = useState<Record<string, string>>(
    {},
  );

  const columnDefs = useMemo((): ColumnDef<Record<string, unknown>>[] => {
    const defs: ColumnDef<Record<string, unknown>>[] = [];

    if (showSelectionCheckbox) {
      defs.push({
        id: "__selection__",
        header: ({ table }) => (
```

```
          <PremiumCheckbox
            checked={table.getIsAllRowsSelected()}
            isIndeterminate={table.getIsSomeRowsSelected()}
            onChange={(checked) => table.toggleAllRowsSelected(!!checked)}
            className="h-4 w-4"
          />
        ),
        cell: ({ row }) => (
          <PremiumCheckbox
            checked={row.getIsSelected()}
            onChange={(checked) => row.toggleSelected(!!checked)}
            className="h-4 w-4"
          />
        ),
        size: 48,
        enableSorting: false,
        enableColumnFilter: false,
        enableResizing: false,
      });
    }

    columns.forEach((col) => {
      defs.push({
        id: col.key,
        accessorKey: col.key,
        header: col.headerLabel,
        size: Math.max(80, (col.widthPercent / 100) * 1200),
        minSize: 60,
        maxSize: 800,
        enableSorting: col.enableSorting !== false,
        enableColumnFilter: col.enableFiltering !== false,
        enableResizing: true,
        enableHiding: true,
      });
    });

    return defs;
  }, [columns, showSelectionCheckbox]);

  const table = useMemo(() => {
    const tableInstance = createTable<Record<string, unknown>>({
      data,
      columns: columnDefs,
      state: {
        sorting,
        columnFilters,
        globalFilter,
        columnOrder,
        columnVisibility,
        rowSelection,
```

```
      columnSizing,
    },
    onSortingChange: setSorting,
    onColumnFiltersChange: setColumnFilters,
    onGlobalFilterChange: setGlobalFilter,
    onColumnOrderChange: setColumnOrder,
    onColumnVisibilityChange: setColumnVisibility,
    onRowSelectionChange: setRowSelection,
    onColumnSizingChange: setColumnSizing,
    getRowId: (row) =>
      String(row[rowIdentifierKey] ?? row.id ?? Math.random()),
    manualSorting: true,
    manualFiltering: true,
    enableRowSelection: true,
    getCoreRowModel: getCoreRowModel(),
    getSortedRowModel: getSortedRowModel(),
    getFilteredRowModel: getFilteredRowModel(),
    onStateChange: () => {},
    renderFallbackValue: () => null,
  });
  return tableInstance;
}, [
  data,
  columnDefs,
  sorting,
  columnFilters,
  globalFilter,
  columnOrder,
  columnVisibility,
  rowSelection,
  columnSizing,
  rowIdentifierKey,
]);

const virtualizer = useVirtualizer({
  count: table.getRowModel().rows.length,
  getScrollElement: () => parentRef.current,
  estimateSize: () => rowHeightPx,
  overscan: 5,
});

const handleEditChange = useCallback((newValue: unknown) => {
  setEditValue(newValue);
}, []);

const handleEditBlur = useCallback(
  (rowIndex: number, columnKey: string, oldValue: unknown) => {
    if (
      editingCell &&
      editingCell.rowIndex === rowIndex &&
```

```
      editingCell.columnKey === columnKey
    ) {
      if (editValue !== oldValue) {
        const updatedData = [...data];
        updatedData[rowIndex] = {
          ...updatedData[rowIndex],
          [columnKey]: editValue,
        };

        onDataChange?.(updatedData, {
          rowIndex,
          columnKey,
          oldValue,
          newValue: editValue,
        });
      }
      setEditingCell(null);
      setEditValue("");
    }
  },
  [editingCell, editValue, data, onDataChange],
);

const handleKeyDown = useCallback(
  (
    e: React.KeyboardEvent,
    rowIndex: number,
    columnKey: string,
    value: unknown,
  ) => {
    if (e.key === "Enter" && !e.shiftKey) {
      e.preventDefault();
      if (!isEditable) return;
      setEditingCell({ rowIndex, columnKey });
      setEditValue(value);
    } else if (e.key === "Escape" && editingCell) {
      setEditingCell(null);
      setEditValue("");
    } else if (e.key === "Tab") {
      if (editingCell) {
        handleEditBlur(
          editingCell.rowIndex,
          editingCell.columnKey,
          data[editingCell.rowIndex]?.[editingCell.columnKey] as unknown,
        );
      }
    }
  },
  [isEditable, editingCell, data, handleEditBlur],
);
```

```
const handleRowClick = useCallback(
  (row: Record<string, unknown>, index: number) => {
    onRowClick?.(row, index);
  },
  [onRowClick],
);

const handleContextMenu = useCallback(
  (e: React.MouseEvent, row: Record<string, unknown>) => {
    e.preventDefault();
    popupMenu?.trigger(e, row);
  },
  [popupMenu],
);

const handleSort = useCallback(
  (column: Column<Record<string, unknown>>) => {
    const isSorted = column.getIsSorted();
    column.toggleSorting(
      isSorted === "asc" ? false : isSorted === "desc" ? undefined : true,
    );
  },
  [],
);

const handleFilterChange = useCallback(
  (columnKey: string, value: string) => {
    setFilterInputs((prev) => ({ ...prev, [columnKey]: value }));
    table.setColumnFilters((prev) => {
      const existing = prev.find((f) => f.id === columnKey);
      if (value === "") {
        return prev.filter((f) => f.id !== columnKey);
      }
      if (existing) {
        return prev.map((f) => (f.id === columnKey ? { ...f, value } : f));
      }
      return [...prev, { id: columnKey, value }];
    });
  },
  [table],
);

const handleColumnResizeStart = useCallback(
  (e: React.MouseEvent, columnKey: string) => {
    e.preventDefault();
    e.stopPropagation();
    const column = table.getColumn(columnKey);
    if (column) {
      setIsResizing({
```

```
        columnKey,
        startX: e.clientX,
        startWidth: column.getSize(),
      });
    }
  },
  [table],
);

const handleColumnResize = useCallback(
  (e: MouseEvent) => {
    if (isResizing) {
      const newWidth = Math.max(
        60,
        isResizing.startWidth + (e.clientX - isResizing.startX),
      );
      table.setColumnSizing((prev) => ({
        ...prev,
        [isResizing.columnKey]: newWidth,
      }));
    }
  },
  [isResizing, table],
);

const handleColumnResizeEnd = useCallback(() => {
  setIsResizing(null);
}, []);

const handleDragStart = useCallback(
  (e: React.DragEvent, columnKey: string) => {
    if (!isColumnMovable) return;
    setDraggedColumn(columnKey);
    e.dataTransfer.effectAllowed = "move";
  },
  [isColumnMovable],
);

const handleDragOver = useCallback((e: React.DragEvent) => {
  e.preventDefault();
  e.dataTransfer.dropEffect = "move";
}, []);

const handleDrop = useCallback(
  (e: React.DragEvent, targetColumnKey: string) => {
    e.preventDefault();
    if (draggedColumn && draggedColumn !== targetColumnKey) {
      const columns = table.getAllColumns();
      const fromIndex = columns.findIndex(
        (c: Column<Record<string, unknown>>) => c.id === draggedColumn,
```

```
      );
      const toIndex = columns.findIndex(
        (c: Column<Record<string, unknown>>) => c.id === targetColumnKey,
      );
      if (fromIndex !== -1 && toIndex !== -1) {
        const newOrder = [...columnOrder];
        const [removed] = newOrder.splice(fromIndex, 1);
        newOrder.splice(toIndex, 0, removed);
        setColumnOrder(newOrder);
        onColumnMove?.(fromIndex, toIndex);
      }
    }
    setDraggedColumn(null);
  },
  [draggedColumn, columnOrder, table, onColumnMove],
);

const handleRowDragStart = useCallback(
  (e: React.DragEvent, rowIndex: number) => {
    if (!isRowMovable) return;
    setDraggedRow(rowIndex);
    e.dataTransfer.effectAllowed = "move";
  },
  [isRowMovable],
);

const handleRowDragOver = useCallback((e: React.DragEvent) => {
  e.preventDefault();
  e.dataTransfer.dropEffect = "move";
}, []);

const handleRowDrop = useCallback(
  (e: React.DragEvent, targetRowIndex: number) => {
    e.preventDefault();
    if (draggedRow !== null && draggedRow !== targetRowIndex) {
      onRowMove?.(draggedRow, targetRowIndex);
    }
    setDraggedRow(null);
  },
  [draggedRow, onRowMove],
);

useEffect(() => {
  const handleMouseMove = (e: MouseEvent) => handleColumnResize(e);
  const handleMouseUp = () => handleColumnResizeEnd();

  if (isResizing) {
    window.addEventListener("mousemove", handleMouseMove);
    window.addEventListener("mouseup", handleMouseUp);
  }
```

```
  return () => {
    window.removeEventListener("mousemove", handleMouseMove);
    window.removeEventListener("mouseup", handleMouseUp);
  };
}, [isResizing, handleColumnResize, handleColumnResizeEnd]);

const sortedColumns = useMemo(() => {
  const allColumns = table.getAllColumns();
  if (columnOrder.length > 0) {
    return columnOrder
      .map((id) =>
        allColumns.find(
          (c: Column<Record<string, unknown>>) => c.id === id,
        ),
      )
      .filter(Boolean) as Column<Record<string, unknown>>[];
  }
  return allColumns;
}, [table, columnOrder]);

const headerGroups = table.getHeaderGroups();
const rows = table.getRowModel().rows;

const getAlignmentClass = (align?: string) => {
  switch (align) {
    case "center":
      return "text-center";
    case "right":
      return "text-right";
    default:
      return "text-left";
  }
};

const renderCellEditor = (
  column: Column<Record<string, unknown>>,
  row: Row<Record<string, unknown>>,
  value: unknown,
) => {
  const colConfig = columns.find((c) => c.key === column.id);
  if (!colConfig || !colConfig.editType) return null;

  const isEditing =
    editingCell?.rowIndex === row.index &&
    editingCell?.columnKey === column.id;

  if (!isEditing) return null;

  const commonProps = {
```

```
    value: editValue,
    onChange: handleEditChange,
    onBlur: () => handleEditBlur(row.index, column.id, value),
    onKeyDown: (e: React.KeyboardEvent) =>
      handleKeyDown(e, row.index, column.id, value),
    className: "w-full h-full min-h-[36px]",
    error: error,
  };

  switch (colConfig.editType) {
    case "text":
      return <TextBox {...commonProps} value={String(editValue ?? "")} />;
    case "numeric":
      return (
        <NumericEdit {...commonProps} value={Number(editValue ?? 0)} />
      );
    case "date":
      return <DateEdit {...commonProps} value={String(editValue ?? "")} />;
    case "checkbox":
      return (
        <PremiumCheckbox
          checked={editValue as boolean}
          onChange={(checked) => handleEditChange(checked)}
          className="h-4 w-4"
        />
      );
    case "combo":
      return (
        <ComboBox
          options={colConfig.editOptions || []}
          value={String(editValue ?? "")}
          onChange={handleEditChange}
          className="w-full h-full min-h-[36px]"
        />
      );
    default:
      return null;
  }
};

const renderCellContent = (
  column: Column<Record<string, unknown>>,
  row: Row<Record<string, unknown>>,
) => {
  const value = row.getValue(column.id);
  const colConfig = columns.find((c) => c.key === column.id);
  const isEditing =
    editingCell?.rowIndex === row.index &&
    editingCell?.columnKey === column.id;
```

```
      if (isEditing) {
        return renderCellEditor(column, row, value);
      }

      if (colConfig?.editType === "checkbox") {
        return (
          <PremiumCheckbox
            checked={value as boolean}
            disabled={!isEditable}
            onChange={
              isEditable
                ? (checked) => {
                    const updatedData = [...data];
                    updatedData[row.index] = {
                      ...updatedData[row.index],
                      [column.id]: checked,
                    };
                    onDataChange?.(updatedData, {
                      rowIndex: row.index,
                      columnKey: column.id,
                      oldValue: value,
                      newValue: checked,
                    });
                  }
                : undefined
            }
            className="h-4 w-4"
          />
        );
      }

      return (
        <div className={cn("truncate", getAlignmentClass(colConfig?.align))}>
          {String(value ?? "")}
        </div>
      );
    };

    return (
      <div
        ref={(el) => {
          parentRef.current = el;
          if (ref) {
            if (typeof ref === "function") ref(el);
            else ref.current = el;
          }
        }}
        className={cn(
          "relative w-full overflow-hidden rounded-lg border
border-[color-mix(in_oklch,var(--color-border)_40%,transparent)] bg-card",
```

```
          className,
        )}
        {...props}
      >
        {hint && (
          <HintBox content={hint} className="mb-1.5">
            <span className="text-sm text-text-muted" />
          </HintBox>
        )}

        {label && (
          <div className="px-4 py-2 border-b
border-[color-mix(in_oklch,var(--color-border)_40%,transparent)] bg-card/40">
            <span className="font-semibold text-text-main">{label}</span>
          </div>
        )}

        <div
          className="relative overflow-auto"
          style={{ maxHeight: "600px" }}
          onKeyDown={(e) => {
            if (e.key === "ArrowDown" || e.key === "ArrowUp") {
              e.preventDefault();
            }
          }}
        >
          <table
            ref={tableRef}
            className="w-full border-collapse"
            style={{
              minWidth: sortedColumns.reduce(
                (sum: number, col: Column<Record<string, unknown>>) =>
                  sum + (col.getSize() || 100),
                0,
              ),
            }}
            role="grid"
          >
            <thead className="sticky top-0 z-10">
              {headerGroups.map(
                (
                  headerGroup: import("@tanstack/table-core").HeaderGroup<
                    Record<string, unknown>
                  >,
                ) => (
                  <tr
                    key={headerGroup.id}
                    className="bg-card/40 border-b
border-[color-mix(in_oklch,var(--color-border)_40%,transparent)]"
                  >
```

```
                    {headerGroup.headers.map(
                      (
                        header: import("@tanstack/table-core").Header<
                          Record<string, unknown>,
                          unknown
                        >,
                      ) => {
                        const column = header.column;
                        const isSorted = column.getIsSorted();
                        const canSort = column.getCanSort();
                        const isResizing = column.getIsResizing();
                        const isDragged = draggedColumn === column.id;

                        return (
                          <th
                            key={column.id}
                            className={cn(
                              "relative px-3 py-2 font-semibold text-text-main
whitespace-nowrap",
                              "border-r
border-[color-mix(in_oklch,var(--color-border)_40%,transparent)]",
                              "last:border-r-0",
                              "select-none",
                              isDragged && "opacity-50",
                              isResizing && "bg-brand-primary/10",
                            )}
                            style={{ width: column.getSize() }}
                            draggable={
                              isColumnMovable && column.id !== "__selection__"
                            }
                            onDragStart={(e) => handleDragStart(e, column.id)}
                            onDragOver={handleDragOver}
                            onDrop={(e) => handleDrop(e, column.id)}
                            onMouseDown={(e) => {
                              if (
                                e.target === e.currentTarget &&
                                column.getCanResize()
                              ) {
                                handleColumnResizeStart(e, column.id);
                              }
                            }}
                          >
                            <div className="flex items-center gap-2">
                              {column.id === "__selection__" ? (
                                typeof header.column.columnDef.header ===
                                "function" ? (
                                  header.column.columnDef.header({
                                    table,
                                    column: header.column,
                                    header: header.column.columnDef.header,
```

```
                                  } as unknown as
import("@tanstack/table-core").HeaderContext<
                                    Record<string, unknown>,
                                    unknown
                                  >)
                                ) : (
                                  header.column.columnDef.header
                                )
                              ) : (
                                <>
                                  {canSort && (
                                    <div
                                      onClick={() => handleSort(column)}
                                      className={cn(
                                        "flex items-center gap-1 p-1 rounded
transition-all duration-100 active:scale-95
hover:bg-[color-mix(in_oklch,var(--color-brand-primary)_10%,transparent)]
cursor-pointer",
                                        isSorted && "text-brand-primary",
                                      )}
                                      aria-label={`Sort by ${column.id}`}
                                      role="button"
                                      tabIndex={0}
                                      onKeyDown={(e) => {
                                        if (
                                          e.key === "Enter" ||
                                          e.key === " "
                                        ) {
                                          e.preventDefault();
                                          handleSort(column);
                                        }
                                      }}
                                    >
                                      {typeof header.column.columnDef.header ===
                                      "function"
                                        ? header.column.columnDef.header({
                                            table,
                                            column: header.column,
                                            header:
                                              header.column.columnDef.header,
                                          } as unknown as
import("@tanstack/table-core").HeaderContext<
                                            Record<string, unknown>,
                                            unknown
                                          >)
                                        : header.column.columnDef.header}
                                      {isSorted === "asc" && (
                                        <ArrowUp className="h-3.5 w-3.5" />
                                      )}
                                      {isSorted === "desc" && (
```

```
                                        <ArrowDown className="h-3.5 w-3.5" />
                                      )}
                                      {!isSorted && (
                                        <ArrowUpDown className="h-3.5 w-3.5
text-text-muted" />
                                      )}
                                    </div>
                                  )}
                                  {!canSort &&
                                    (typeof header.column.columnDef.header ===
                                    "function"
                                      ? header.column.columnDef.header({
                                          table,
                                          column: header.column,
                                          header:
                                            header.column.columnDef.header,
                                        } as unknown as
import("@tanstack/table-core").HeaderContext<
                                          Record<string, unknown>,
                                          unknown
                                        >)
                                      : header.column.columnDef.header)}
                                </>
                              )}
                              {column.getCanFilter() &&
                                column.id !== "__selection__" && (
                                  <div className="relative flex-1 min-w-0">
                                    <input
                                      type="text"
                                      placeholder="Filter..."
                                      value={filterInputs[column.id] || ""}
                                      onChange={(e) =>
                                        handleFilterChange(
                                          column.id,
                                          e.target.value,
                                        )
                                      }
                                      onClick={(e) => e.stopPropagation()}
                                      className="w-full px-2 py-1 text-xs
bg-background border
border-[color-mix(in_oklch,var(--color-border)_40%,transparent)] rounded
focus:outline-none focus:ring-1 focus:ring-brand-primary"
                                    />
                                  </div>
                                )}
                            </div>
                            {column.getCanResize() && (
                              <div
                                className="absolute right-0 top-0 h-full w-1
cursor-col-resize hover:w-2 hover:bg-brand-primary/30 transition-all"
```

```
                                onMouseDown={(e) =>
                                  handleColumnResizeStart(e, column.id)
                                }
                              />
                            )}
                          </th>
                        );
                      },
                    )}
                  </tr>
                ),
              )}
            </thead>
            <tbody>
              {virtualizer.getVirtualItems().map((virtualRow) => {
                const row = rows[virtualRow.index];
                const isSelected = row.getIsSelected();
                const isActive =
                  activeRowKey &&
                  row.getValue(rowIdentifierKey) === activeRowKey;
                const isEven = virtualRow.index % 2 === 0;

                return (
                  <tr
                    key={row.id}
                    className={cn(
                      "transition-colors duration-150",
                      "hover:bg-brand-primary/3",
                      isSelected &&
                        "bg-brand-primary/5 text-text-main font-semibold border-l-2
border-brand-primary",
                      isActive && "bg-brand-primary/10",
                      isEven &&

"bg-[color-mix(in_oklch,var(--color-border)_5%,transparent)]",
                      "border-b
border-[color-mix(in_oklch,var(--color-border)_20%,transparent)]",
                      "last:border-b-0",
                    )}
                    style={{
                      height: rowHeightPx,
                      transform: `translateY(${virtualRow.start}px)`,
                    }}
                    draggable={isRowMovable}
                    onDragStart={(e) => handleRowDragStart(e, virtualRow.index)}
                    onDragOver={handleRowDragOver}
                    onDrop={(e) => handleRowDrop(e, virtualRow.index)}
                    onClick={() =>
                      handleRowClick(row.original, virtualRow.index)
                    }
```

```
                    onContextMenu={(e) => handleContextMenu(e, row.original)}
                  >
                    {sortedColumns.map((column) => (
                      <td
                        key={column.id}
                        className={cn(
                          "px-3 py-2 whitespace-nowrap overflow-hidden",
                          "border-r
border-[color-mix(in_oklch,var(--color-border)_20%,transparent)]",
                          "last:border-r-0",
                          getAlignmentClass(
                            columns.find((c) => c.key === column.id)?.align,
                          ),
                        )}
                        style={{ width: column.getSize() }}
                      >
                        {renderCellContent(column, row)}
                      </td>
                    ))}
                  </tr>
                );
              })}
              {rows.length === 0 && (
                <tr>
                  <td
                    colSpan={sortedColumns.length}
                    className="px-4 py-8 text-center text-text-muted"
                  >
                    No data available
                  </td>
                </tr>
              )}
            </tbody>
          </table>
        </div>

        <div className="border-t
border-[color-mix(in_oklch,var(--color-border)_40%,transparent)] p-3 flex
justify-between items-center bg-card/40 shrink-0">
          <div className="text-xs text-text-muted">
            Showing {table.getRowModel().rows.length} records
          </div>
          <StandalonePagination
            currentPage={table.getState().pagination.pageIndex + 1}
            totalPages={table.getPageCount()}
            onPageChange={(targetPage) => table.setPageIndex(targetPage - 1)}
            disabled={props.disabled}
          />
        </div>
```

```
        {error && (
          <div className="px-4 py-2 border-t
border-[color-mix(in_oklch,var(--color-border)_40%,transparent)] bg-destructive/5">
            <span className="text-sm text-destructive">{error}</span>
          </div>
        )}
      </div>
    );
  },
);
Grid.displayName = "Grid";
```